5 주차 보고서

전공: 컴퓨터공학과 학년: 4 학년 학번: 20232019 이름: Maksutova Aibike

Problem 1: Who is the Richest?

1. Idea

We can only ask "does person i have at least x won?". With this question we can do binary search on one person's money. Money is at most 10^9, and 2^30 > 10^9, so we need about 30 questions to know the exact money of one person.

The simple way is to do binary search for every person and take the biggest. But this needs 30 * N questions, which is too many. We don't really need to know the exact money of everyone. We only need the exact money of the richest person.

So my idea is to keep the current richest person (I call him the champion) and his exact money V. For every other person I ask only one question: "do you have at least V+1 won?". If the answer is No, this person is not richer, so I skip him. If the answer is Yes, he is the new champion, and I do binary search only for him to find his exact money.

2. Algorithm

1. Shuffle the people in random order.
2. Do binary search on the first person in [0, 10^9]. Now he is the champion and V = his money.
3. For each next person i, ask "at least V+1?".
   - No: skip person i.
   - Yes: binary search person i in [V+1, 10^9], then champion = i, V = his money.
4. At the end print the champion and V.

```
best = p[0]; V = binary_search(p[0], 0, 1e9)
for i in p[1..N-1]:
    if ask(i, V+1) == Yes:
        best = i; V = binary_search(i, V+1, 1e9)
print(best, V)
```

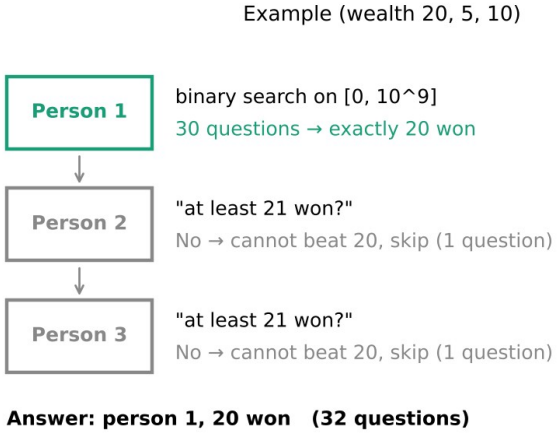


Figure 1. How the algorithm works on the example.

In the example, person 1 needs 30 questions (we get 20 won). Then for person 2 and person 3 we ask "at least 21?" and both say No. So the answer is person 1 with 20 won, and we used only 32 questions instead of 90.

This is correct because a No answer means the person has at most V won, so he can't be richer than the champion. A Yes answer means he has more than V, so he becomes the new richest. If two people have the same money, we keep the first one, and the problem says any of them is OK.

3. Why shuffle?

Most people cost only 1 question. Only when the champion changes we pay about 30 more questions. If the people are sorted by money from small to big, the champion changes every time and we again need about 30 * N questions. If we shuffle first, the k-th person is the richest so far only with probability 1/k, so the champion changes only about ln N times on average. So the total is about N + 30 * ln N questions. For example, for N = 100000 it is around 100 360 questions, compared with 3 000 000 for the simple way.

We can't do much better than this, because every person except the answer must be asked at least once (otherwise maybe he is the richest), and we also need around 30 questions to find the exact amount. So about N + 30 is the minimum.

Time complexity: O(N + 30 log N) on average. Memory: O(N).

Problem 2: Chasing Light

1. What the problem asks

Each line of the input is one light: X Y S E. It means a light is on at cell (X, Y) from time S to time E. The robot starts at (0, 0) at time 0. Every second it can do one of two things:

- move 1 cell (up, down, left or right), or
- stay. If the cell it stays on has light at that second, it gets 1 energy.

So the energy is simply the number of seconds the robot is standing under a light. Time spent walking gives nothing. To walk from (x1, y1) to (x2, y2) the robot needs |x1 - x2| + |y1 - y2| seconds. A light from S to E can give at most E - S energy (for example, light [0, 4] gives 4).

Example input: L1 = (0,0) on [0,4], L2 = (4,0) on [5,8], L3 = (0,3) on [5,9], L4 = (1,3) on [10,13].

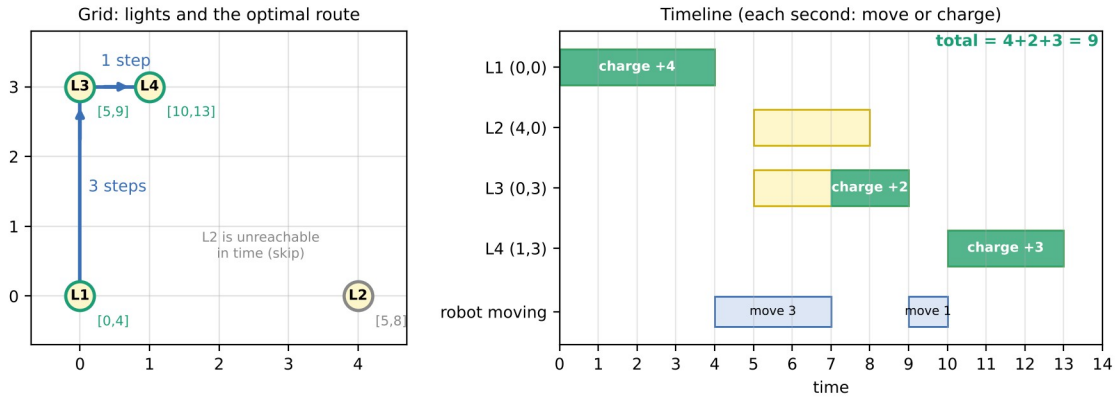


Figure 2. Best route for the example: 4 + 2 + 3 = 9.

Best plan for the example (Figure 2):

- time 0~4: stay at L1, +4 energy
- time 4~7: walk 3 cells to L3 (0,3)
- time 7~9: stay at L3, +2 energy (L3 turns off at 9)
- time 9~10: walk 1 cell to L4 (1,3)
- time 10~13: stay at L4, +3 energy

Total = 4 + 2 + 3 = 9. L2 is skipped: it is far from the others, so going there wastes too much time.

2. Main idea: stay until the light turns off

When the robot is standing under a light, there is no reason to leave before that light turns off. Look at Figure 3. Light j is on until 13, the next light k is on from 10, and walking from j to k takes 5 seconds.

(a) Leave j at time 8: 6 energy at j, arrive at k at 13, 7 energy at k. Total 13.

(b) Leave j at time 10: 8 energy at j, arrive at k at 15, 5 energy at k. Total 13.

Every extra second at j gives +1, and then we lose at most 1 second at the next light. If the next light is not on yet when we arrive, we lose nothing. So staying longer is never worse.

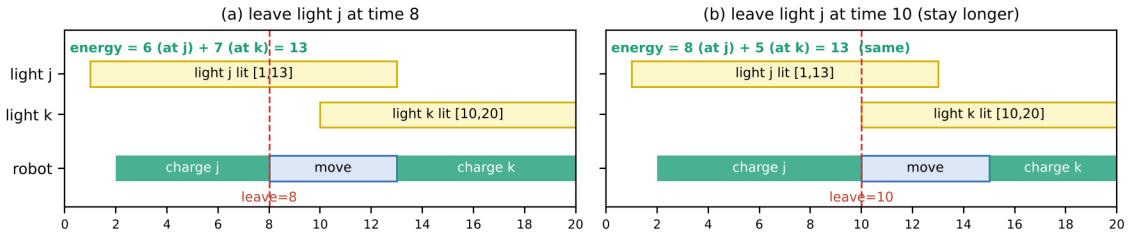


Figure 3. Staying 2 seconds longer at light j does not make the total smaller.

This makes the problem much easier. The robot only needs to choose which lights to visit and in which order. At every chosen light it stays until the light turns off, then walks to the next one.

If the robot arrives at light j at time a, it gets E_j - max(S_j, a) energy there:

- if it comes early (a < S_j), it waits for the light and gets the full E_j - S_j;
- if it comes late (a > S_j), it gets only E_j - a;
- if a > E_j, the light is already off and we can't use it.

3. DP

dp[j] = the maximum energy the robot can have when it finishes light j (it is at light j at time E_j).

How to compute dp[j]: try every light i that the robot could have finished just before j. The robot leaves i at time E_i and arrives at j at time a = E_i + dist(i, j). If a <= E_j, then

dp[j] = max(dp[j], dp[i] + E_j - max(S_j, a))

Steps:

1. Add a fake light "start" at (0, 0) with S = E = 0 and dp[start] = 0.
2. Sort all lights by E (end time), so when we compute dp[j], all lights that end earlier are already done.
3. For each light j, check all lights i before it with the formula above.
4. The answer is the biggest dp value.

Example (lights sorted by E: L1, L2, L3, L4):

light j	try from	arrival a	energy at j	dp
L1 (0,0) [0,4]	start	0	4 - 0 = 4	4
L2 (4,0) [5,8]	start / L1	4 / 8	3 / 0	max(0+3, 4+0) = 4
L3 (0,3) [5,9]	start / L1	3 / 7	4 / 2	max(0+4, 4+2) = 6
L4 (1,3) [10,13]	L1 / L3	8 / 10	3 / 3	max(4+3, 6+3) = 9

(For L4, coming from start or from L2 is worse or impossible: from L2 the robot arrives at 14, after L4 is off.) The answer is dp[L4] = 9, the same as in the problem. The path is start -> L1 -> L3 -> L4.

4. Complexity

Sorting takes O(N log N). For every light j we check all lights i before it, so the DP is O(N^2). Memory is O(N). I also wrote a brute force that simulates every second and compared it with the DP on many small random tests. The answers were always the same.